

Clinic Appointment & Patient Records API

Improvement Report

Spring Boot · REST API · Java

TRAINEE Safa Ahmed ALSibani	DATE 25 June 2026
PROJECT Clinic Appointment & Patient Records API	STACK Spring Boot / Java / H2 / Postman

Table of Contents

01	Introduction	3
02	Challenges Faced	3
03	Issues and Bugs Identified	3
04	How the Issues Were Resolved	4
05	Technical Decisions	4
06	Suggestions for Future Versions	5
07	Lessons Learned	5
08	Conclusion	5
09	Screenshots & Testing Evidence	6
10	Screenshots & Testing Evidence	7
11	Screenshots & Testing Evidence	8

01 Introduction

During the team project, our group was assigned an existing Spring Boot application for managing doctors, patients, appointment slots, appointments, and patient visit records. The objective was to analyze the project, identify issues, improve the implementation, and ensure that the system was functional and properly documented.

02 Challenges Faced

One of the main challenges was understanding the existing project structure and business logic before making modifications. The project contained multiple entities and relationships:

- ◆ Doctors & Patients
- ◆ Appointment Slots & Appointments
- ◆ Patient Visit Records

Another challenge was ensuring that new improvements did not break existing functionality. Each modification was tested carefully using Postman and the H2 database.

03 Issues and Bugs Identified

■ Missing Input Validation

Several request DTOs accepted invalid or incomplete data:

- ◆ Empty or null names
- ◆ Invalid phone number formats
- ◆ Future dates of birth
- ◆ Missing required fields

This could allow incorrect or malformed data to be stored in the database.

■ No Doctor Search Functionality

The doctor endpoint returned all doctors without supporting search or filtering. Users could not filter doctors by name or specialty.

■ Inconsistent Doctor Specialties

Doctor specialties were stored as free-text values, leading to inconsistent records:

- ◆ "General Medicine"
- ◆ "general medicine"
- ◆ "General medicine" ← same specialty, three spellings

04 How the Issues Were Resolved

■ Input Validation Improvements

Validation annotations were added to all request DTOs:

- ◆ @NotNull
- ◆ @NotBlank
- ◆ @Size
- ◆ @Pattern
- ◆ @Past

A global validation response format was implemented to provide clear, consistent error messages.

■ Doctor Search and Filtering

The doctor listing endpoint was enhanced to support filtering by name and specialty. Repository and service methods were updated, and API documentation was updated accordingly.

■ Specialty Standardization

A Specialty enum was introduced to define a fixed list of valid specialties. The Doctor entity and DTOs were updated to use this enum instead of free-text values, guaranteeing consistency across all doctor records.

05 Technical Decisions

1. Use Bean Validation annotations to simplify request validation.
2. Use Spring Data JPA query methods for search functionality.
3. Use an Enum for doctor specialties to maintain consistent data.
4. Return meaningful validation messages for a better developer experience.
5. Preserve existing business rules while introducing improvements.

These decisions improved maintainability, data quality, and usability without changing the core system behavior.

06 Suggestions for Future Versions

- Add Swagger / OpenAPI documentation for interactive API exploration.
- Implement authentication and authorization (e.g. Spring Security + JWT).
- Add sorting options to all search endpoints.
- Introduce unit and integration testing (JUnit 5, Mockito).
- Add audit fields: createdAt, updatedAt.
- Implement soft delete where appropriate.
- Standardize API response structure across all endpoints.

07 Lessons Learned

- **Input validation** is essential for maintaining data quality.
- **Enum-based standardization** prevents inconsistent records.
- **Filtering and search** dramatically improve API usability.
- **Testing after every change** safeguards existing functionality.
- **Good documentation** helps future developers understand the system faster.

08 Conclusion

The project was successfully analyzed and improved by identifying weaknesses, implementing validation enhancements, adding doctor search functionality, and standardizing doctor specialties. These changes improved data consistency, user experience, and overall system quality while maintaining the original functionality of the application.

09 Screenshots & Testing Evidence

The following screenshots demonstrate the implemented features tested via Postman against the running Spring Boot application.

1) Input Validation

POST <http://localhost:8080/api/patients>

The API validates patient input data before saving it. Invalid values such as an empty name, a future date of birth, and an incorrect phone number are rejected with clear validation messages.

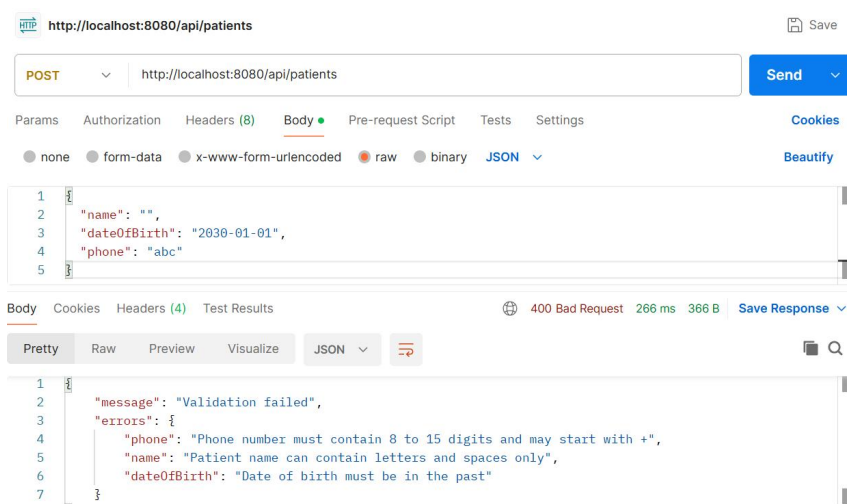


Figure 1 – Input Validation: Invalid patient data rejected with clear error messages (400 Bad Request)

2) Required Fields Validation

POST <http://localhost:8080/api/patients>

Required fields are validated using Bean Validation annotations. Requests with missing mandatory fields are rejected and appropriate error messages are returned.

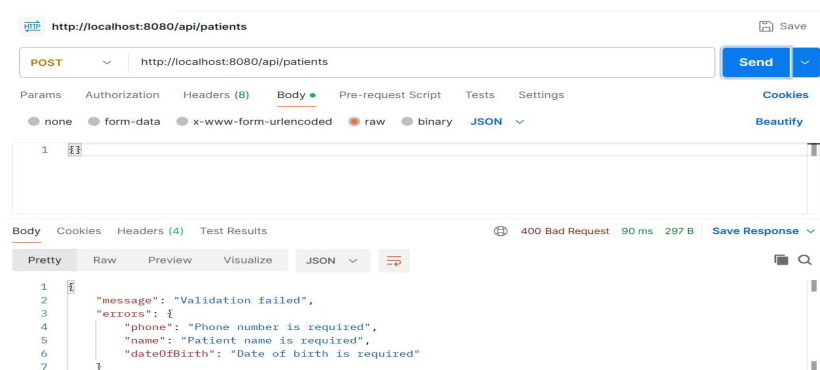


Figure 2 – Required Fields Validation: Empty request body returns validation errors for all mandatory fields

3) Doctor Search by Name

GET <http://localhost:8080/api/doctors?name=Sara>

The doctor endpoint supports filtering by name. Only doctors whose names match the provided search value are returned.

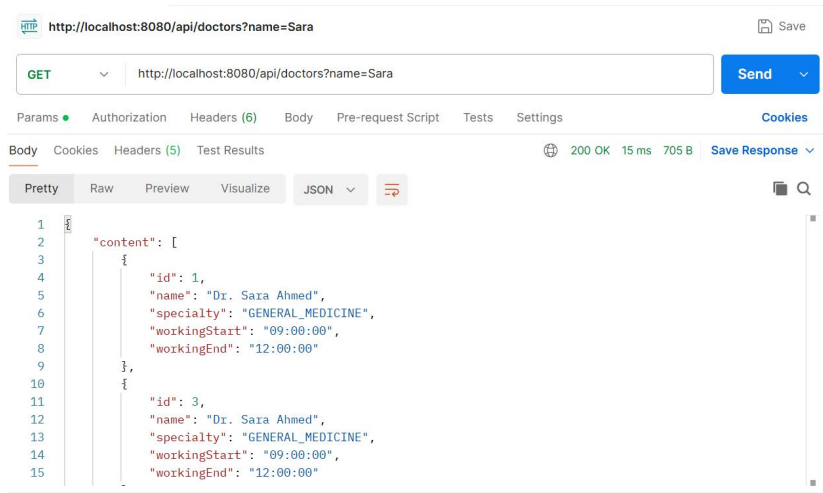


Figure 3 – Doctor Search by Name (response): Paginated results returned for matching doctors

4) Doctor Search by Specialty

GET <http://localhost:8080/api/doctors?specialty=TESTING>

The doctor endpoint supports filtering by specialty. The API returns only doctors that belong to the requested specialty.

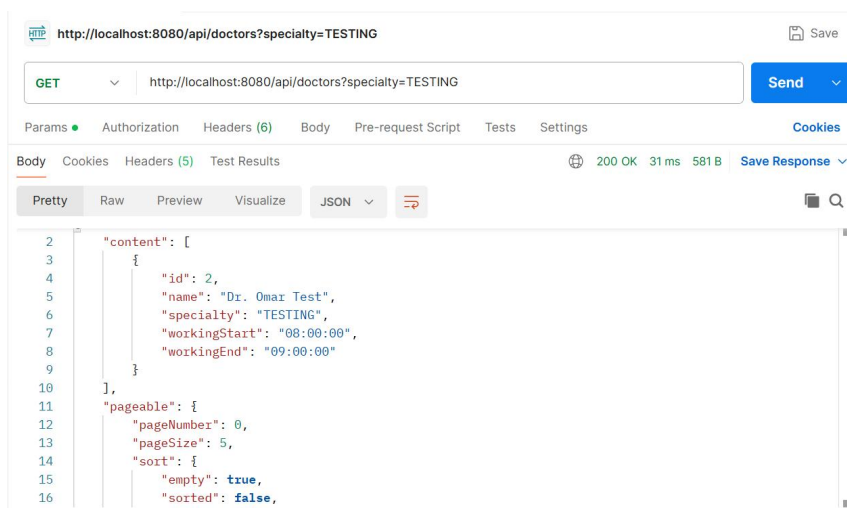


Figure 4 – Doctor Search by Specialty: Filtering doctors by specialty=TESTING returns matching records

5) Invalid Specialty Rejected

POST <http://localhost:8080/api/doctors>

specialties are standardized using a predefined enum. Invalid specialty values are rejected and the API returns the list of allowed values.

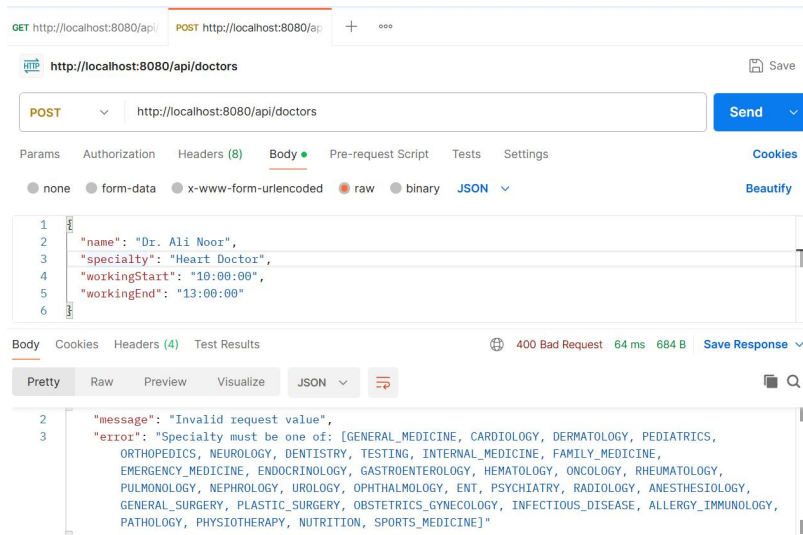


Figure 5 – Invalid Specialty Rejected: POST with unknown specialty returns 400 with allowed values list

6) Valid Specialty Accepted

POST <http://localhost:8080/api/doctors>

The API accepts valid specialty values defined in the Specialty enum. This ensures consistent doctor specialty records across the system.

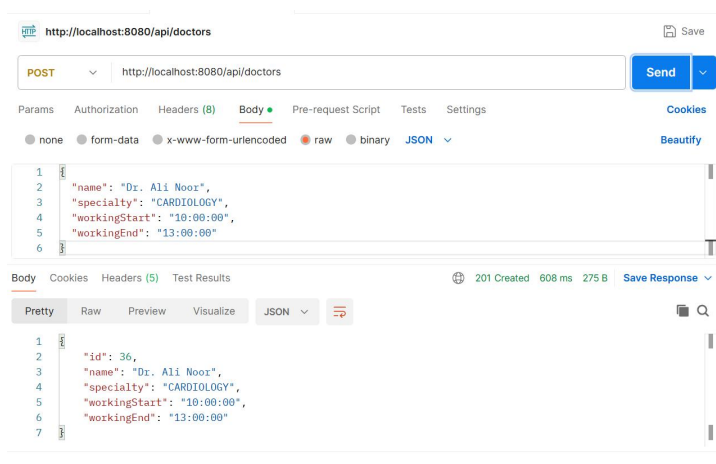


Figure 6 – Valid Specialty Accepted: POST with CARDIOLOGY specialty returns 201 Created